

DevOps. Automatización y monitorización de infraestructuras

Unidad 05. Automatización y monitorización en Odoo - Laboratorio 01



Autor: Sergi García Barea

Actualizado Mayo 2026



Licencia



Reconocimiento – NoComercial – CompartirIgual (BY-NC-SA):

No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

Unidad 05. Automatización y monitorización en Odoo - Laboratorio 01	2
1. Explicación ¿Qué haremos en este caso práctico?	2
2. Objetivos	3
3. El modelo mental	3
4. Topología del proyecto	4
5. Fase de ejecución: despliegue paso a paso	5
Paso 1: Orquestación de red (El cimiento)	5
¿Qué ocurre bajo el capó?	5
Adaptación a Bajos recursos:	5
Paso 2: Infraestructura vía Terraform	5
¿Qué ocurre bajo el capó?	5
Adaptación a bajos recursos:	6
Paso 3: Inicialización de la persistencia (Base de datos)	6
¿Qué ocurre bajo el capó?	6
Adaptación a bajos recursos:	7
Paso 4: Gestión de Configuración con SaltStack (El capataz)	7
¿Qué ocurre bajo el capó?	7
Adaptación a bajos recursos:	7
Paso 5: Verificación de la observabilidad (El dashboard)	7
Paso 6: (OPTATIVO) Ingeniería de caos (Simulación de incidentes)	8
Paso 7: Verificación integral del estado de todos los servicios (auditoría final)	9
¿Qué ocurre bajo el capó?	9
Interpretación de los resultados:	10
Adaptación a Bajos recursos:	10
Verificación final:	10
6. Pruebas manuales	11
7. Operaciones de limpieza	11
8. Problemas comunes	12

UNIDAD 05. AUTOMATIZACIÓN Y MONITORIZACIÓN EN ODOO - LABORATORIO 01

1. EXPLICACIÓN ¿QUÉ HAREMOS EN ESTE CASO PRÁCTICO?

En este laboratorio vamos a construir un entorno de producción simulado para una aplicación crítica de negocio: Odoo ERP 17. A diferencia de unidades anteriores (donde gestionábamos máquinas virtuales pesadas), aquí orquestaremos un conjunto de contenedores Docker mediante Terraform. El objetivo es experimentar con una arquitectura completa de automatización y observabilidad:

- Dos instancias de Odoo (opcionalmente una sola) cada una con su propia base de datos PostgreSQL.
- SaltStack como herramienta de gestión de configuración (arquitectura Master-Minion).
- Stack de observabilidad: Prometheus (métricas), Grafana (visualización), Loki + Alloy (logs centralizados), cAdvisor (monitorización de contenedores) y Blackbox Exporter (sondeos externos).

Nota sobre “Alta Disponibilidad”

En este laboratorio desplegamos dos instancias independientes de Odoo con sus propios datos (no comparten base de datos ni sesiones). No se trata de un clúster de alta disponibilidad real, sino de una forma de enriquecer los paneles de monitorización y practicar con múltiples nodos.

Requisitos del sistema (versión completa – 11 contenedores)

Memoria RAM: 12 GB totales recomendados (el consumo real en pico puede superar los 8 GB, especialmente durante el arranque y la simulación de caos). Con 8 GB físicos y otros procesos del sistema, aunque se puede, es fácil que el OOM Killer actúe.

Alternativa para equipos con menos recursos (Bajos recursos)

Si tu ordenador tiene 8 GB de RAM totales (y libres apenas 3-4 GB), no intentes levantar los 11 contenedores – el sistema colapsará. En su lugar, elige una de estas dos vías antes de ejecutar el despliegue:

A **Opción recomendada: utiliza la rama “Bajos recurso” del laboratorio**

En los materiales encontrarás una versión del laboratorio para “Bajos recursos” (o las instrucciones para modificarlo). En esa versión, se aplican estos cambios:

B **Opción para entornos muy ajustados (4 GB libres o menos)**

Además de los pasos anteriores, puedes:

- Eliminar cAdvisor (ahorro ~256 MB).
- Reducir la memoria de Prometheus de 512 a 256 MB.

El laboratorio seguirá siendo funcional, pero perderás algunas métricas de contenedores y la capacidad de almacenar largos periodos de datos.

2. OBJETIVOS

Al terminar este caso práctico, habrás logrado:

- **Orquestación avanzada con Terraform:** Desplegar una infraestructura multi-contenedor (11 servicios o una versión reducida) de forma declarativa, gestionando redes, volúmenes persistentes y dependencias entre servicios.
- **Gestión centralizada con SaltStack:** Comprender la arquitectura Master-Minion y ser capaz de aceptar llaves criptográficas y gestionar el sistema mediante comandos Salt.
- **Observabilidad:**
 - **Métricas Whitebox** (estado interno de Odoo: usuarios conectados, ventas, latencia de base de datos) mediante un exporter Python personalizado.
 - **Métricas Blackbox** (disponibilidad externa) con Blackbox Exporter y Prometheus.
 - **Métricas de contenedores** con cAdvisor (CPU, RAM, red).
 - **Logs centralizados** (solo en la versión completa) mediante Loki y Alloy, **sin necesidad de instalar agentes dentro de los contenedores de Odoo** (Alloy actúa desde fuera, leyendo la salida estándar a través del socket de Docker).

 **Nota sobre logs centralizados:** La frase "sin agentes intrusivos" significa que **no necesitas instalar un agente de logs dentro de cada contenedor de Odoo**. Alloy sigue siendo un agente, pero se ejecuta una sola vez en un contenedor aparte, escuchando el socket de Docker. En la versión "Bajos recursos" este componente se elimina para ahorrar recursos.

3. EL MODELO MENTAL

Para entender cómo encajan los nodos de este ecosistema (tanto en la versión completa como en la de "**Bajos recursos**"), imagina que estás diseñando el **centro logístico de una gran corporación**. La metáfora se adapta según los recursos disponibles:

La nave industrial (Terraform)

Define el espacio físico: cuántos almacenes (contenedores) se levantan, prepara los muelles de carga (puertos expuestos) y construye la carretera privada (red LabCEFIRE) que los une.

En "Bajos recursos": construyes una nave más pequeña con menos almacenes (por ejemplo, un solo Odoo, sin Loki/Alloy).

El sistema de gestión (Odoo + PostgreSQL)

Es la actividad principal de la empresa. En la versión completa tenemos **dos líneas de montaje gemelas** (dos instancias Odoo + dos bases de datos independientes). **No comparten datos** – no es un clúster de alta disponibilidad real, sino dos instalaciones separadas para enriquecer la monitorización.

En "Bajos recursos": solo una línea de montaje.

El jefe de planta (Salt Master)

Supervisa a los operarios (Minions) desde una torre central. Cada Minion debe llevar una tarjeta de identificación criptográfica (llave RSA). El jefe no da órdenes hasta que no acepta explícitamente esas tarjetas.

En “Bajos recursos”: Funciona igual en ambas versiones (aunque con menos nodos que monitorizar).

El panel de seguridad (Prometheus + Grafana)

Son las cámaras y sensores térmicos que miden el ritmo de trabajo. Cada 15-20 segundos toman una instantánea (scrapeo). Si una línea se atasca (CPU al 100%), el panel gigante de la oficina del director se pone en rojo al instante.

En “Bajos recursos”: Funciona igual en ambas versiones (aunque con menos nodos que monitorizar).

El diario de operaciones (Loki + Alloy)

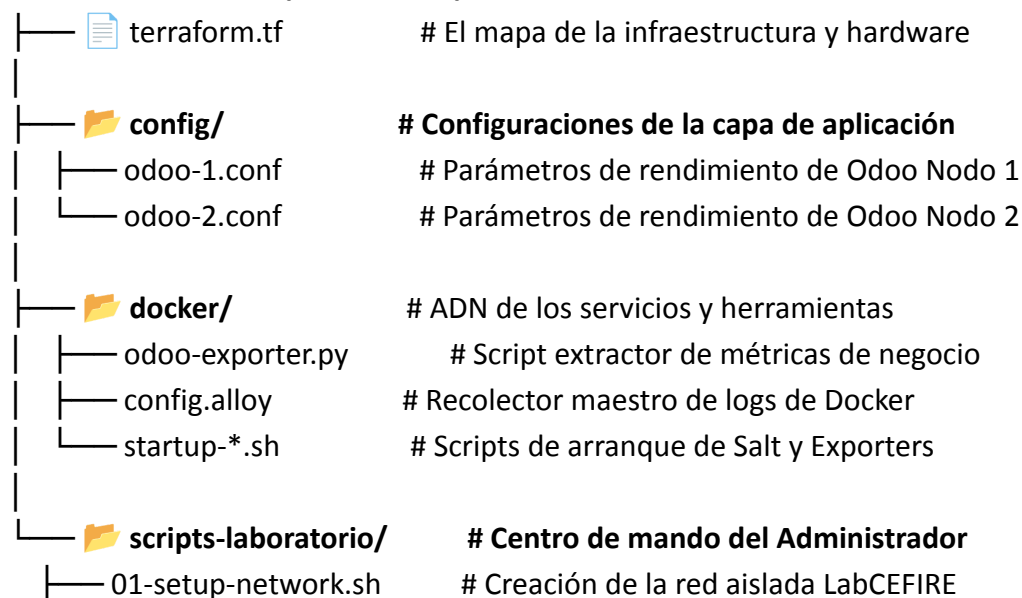
Un perito (Alloy) que lee los paneles de mandos de todas las máquinas (la salida estándar de los contenedores) y los anota en un archivo central (Loki). Cuando ocurre un accidente, puedes ir al archivo y buscar la causa.

En “Bajos recursos”: no contratas al perito (no despliegas Loki/Alloy) para ahorrar recursos. Los registros los consultas directamente con **docker logs**.

4. TOPOLOGÍA DEL PROYECTO

Para mantener la cordura en un despliegue de 11 nodos, la estructura de archivos es vital. Esta es la jerarquía que vamos a utilizar:

Unidad-05-Salt-Odoo/Laboratorio/



- |— 02-deploy-odoo-containers.sh # Invocación de Terraform
- |— 03-check-status-odoo.sh # Comprobamos que Odoo funciona correctamente
- |— 04-create-odoo-database.sh # Inicializamos la base de datos de Odoo
- |— 05-salt-accept-keys.sh # Aceptación de confianza criptográfica
- |— 06-simulate-incidents.sh # Simulador de Ingeniería de Caos
- |— 99-destroy-all.sh # Desmantelamiento total y limpieza

5. 🚀 FASE DE EJECUCIÓN: DESPLIEGUE PASO A PASO

Paso 1: Orquestación de red (El cimiento)

Creamos la red aislada donde vivirán todos los contenedores:

```
bash scripts-laboratorio/01-setup-network.sh
```

🧠 ¿Qué ocurre bajo el capó?

```
alumno@alumno-virtualbox: ~/Escritorio/Unidad-05-Integracion-Salt-Odoo/Laboratorio$ bash scripts-laboratorio/01-setup-network.sh
UD05 - Configuración de Red
[RED] Creando red 'LabCE FIRE' (172.28.0.0/24)...
16c5ca3c582537ddeb2893485d205alc3af02c0b051072ea9f876c7de256fc
✓ Red creada con éxito.
16c5ca3c5825 LabCE FIRE bridge local
```

El script ejecuta ***docker network create LabCE FIRE --subnet=172.28.0.0/24***. Así obtenemos una red virtual privada que nos permite asignar IPs estáticas a cada nodo (ej. 172.28.0.50 para Salt Master, 172.28.0.57 para Odoo). Esto garantiza una comunicación predecible sin depender de la resolución DNS dinámica de Docker.

📊 Adaptación a Bajos recursos:

En la versión de bajos recursos (disponible en un ZIP separado) la red se crea exactamente igual. No necesitas cambiar nada. La red LabCE FIRE sigue siendo la misma, y su creación no penaliza el consumo de RAM. Ejecuta el script sin modificaciones.

Paso 2: Infraestructura vía Terraform

Lanzamos el orquestador para levantar el ecosistema completo:

```
bash scripts-laboratorio/02-deploy-odoo-containers.sh
```

🧠 ¿Qué ocurre bajo el capó?

```
alumno@alumno-virtualbox: ~/Escritorio/Unidad-05-Integracion-Salt-Odoo/Laboratorio$ bash scripts-laboratorio/02-deploy-odoo-containers.sh
UD05 - Despliegue con Terraform
[TF] Inicializando Terraform...
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of kreuzwerker/docker from the dependency lock file
- Installing kreuzwerker/docker v3.9.0...
```

El script ejecuta ***terraform apply -auto-approve*** dentro de la carpeta terraform/. Terraform:

- Reserva la memoria estricta para cada contenedor (ej. 512 MB para Salt Master, 1.5 GB para Odoo).
- Monta volúmenes persistentes (PostgreSQL, Prometheus, Grafana).
- Inyecta los scripts startup-*.sh mediante la directiva upload.
- Bootstrapping asíncrono: Los scripts lanzan la aplicación principal (Odoo, PostgreSQL) de inmediato, mientras que en segundo plano instalan **node_exporter** y (si corresponde) el agente salt-minion, sin bloquear el arranque.

🟡 **Paciencia:** Espera entre 3 y 5 minutos antes de continuar. Los contenedores necesitan descargar dependencias y estabilizar las bases de datos.

Adaptación a bajos recursos:

Si utilizas el laboratorio de bajos recursos, el archivo **terraform.tf** ya viene preconfigurado con las siguientes optimizaciones (no necesitas editar nada):

- Solo un nodo Odoo (se ha eliminado pg2 y **odoo_minion_2**). El laboratorio sigue siendo funcional y el Salt Minion correspondiente sigue presente.
- Sin **Loki** ni **Alloy** (se han comentado esos contenedores), por lo que no hay logs centralizados.
- Memoria de Odoo reducida a 1024 MB (en lugar de 1536 MB).

Ejecuta el script **02-deploy-odoo-containers.sh** sin más cambios. El despliegue consumirá aproximadamente 3-4 GB menos de RAM que la versión completa.

Paso 3: Inicialización de la persistencia (Base de datos)

Odoo no crea las tablas por sí solo al arrancar. Forzamos la creación de la base de datos con datos de demostración:

```
bash scripts-laboratorio/04-create-odoo-database.sh
```

🧠 ¿Qué ocurre bajo el capó?

```
alumno@alumno-virtualbox:~/Escritorio/Unidad-05-Integracion-Salt-Odoo/Laboratorio$ bash scripts-laboratorio/04-create-odoo-database.sh

=====
ODOO DATABASE AUTO CREATOR (JSON-RPC)
=====

Creando BD en Odoo 1 (172.28.0.57:8069)

[1/6] Configurando PostgreSQL...
[OK] PostgreSQL listo
[2/6] Esperando Odoo...
[OK] Odoo responde (HTTP 200)
[3/6] Verificando BD...
[OK] BD no existe
[4/6] Creando base de datos con demo (JSON-RPC)...
```

El script se conecta a cada contenedor Odoo y ejecuta un comando similar a:

```
docker exec odoo-minion-1 odoo --db_host=172.28.0.59 --db_user=odoo
--db_password=odoo -i base --stop-after-init --no-http
```

Así se bypassa el asistente web y se crea la base de datos **odoo_db** incluyendo los datos de

demostración (Demo Data). Sin estos datos, el exporter `odoo-exporter.py` no tendría usuarios, ventas ni facturas que leer, y los paneles de negocio en Grafana mostrarían cero.

🟡 **Paciencia:** La inicialización puede tardar varios minutos por base de datos.

📊 Adaptación a bajos recursos:

El script **`04-create-odoo-database.sh`** del laboratorio de bajos recursos ya está adaptado para inicializar únicamente el nodo `odoo-minion-1`. No necesita comentar ninguna línea. Ejecútalo sin cambios; la base de datos se creará con datos demo en la única instancia de Odoo.

Paso 4: Gestión de Configuración con SaltStack (El capataz)

Autorizamos a los Minions para que el Master pueda controlarlos:

```
bash scripts-laboratorio/05-salt-accept-keys.sh
```

🧠 ¿Qué ocurre bajo el capó?

Aunque el sistema está montado para que, al desplegarse se unan automáticamente al Master. El script ejecuta **`docker exec ud05-salt-master salt-key -A -y`**. Salt opera bajo Zero Trust: aunque los Minions ya están enviando sus llaves RSA, el Master las rechaza hasta que el administrador las acepte explícitamente. Este “apretón de manos” criptográfico establece un canal seguro.

Auditoría: Una vez aceptadas las llaves, lanza:

```
docker exec ud05-salt-master salt '*' test.ping
```

Este script envía un `salt '*' test.ping`. Si todo va bien, cada Minion responde True.

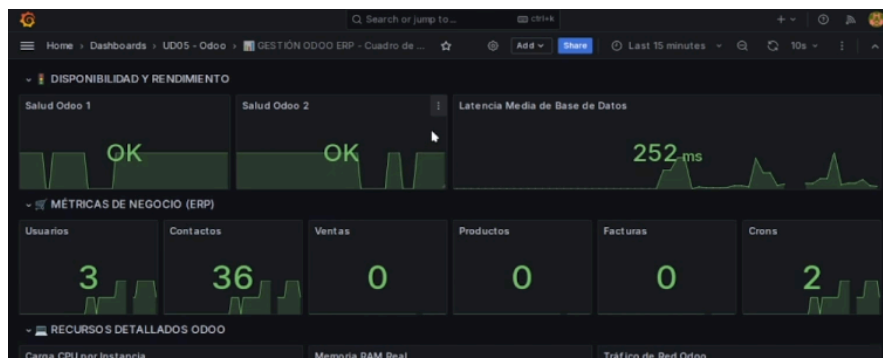
📊 Adaptación a bajos recursos:

El laboratorio de bajos recursos incluye un solo Minion (el de la instancia única de Odoo). El procedimiento es el mismo:

- Acepta las llaves con **`05-salt-accept-keys.sh`** (solo aparecerá una clave).
- Ejecuta **`docker exec ud05-salt-master salt '*' test.ping`** y verás la respuesta True del Minion activo. No es necesario modificar los scripts.

Paso 5: Verificación de la observabilidad (El dashboard)

Validamos que las métricas y logs fluyen correctamente.

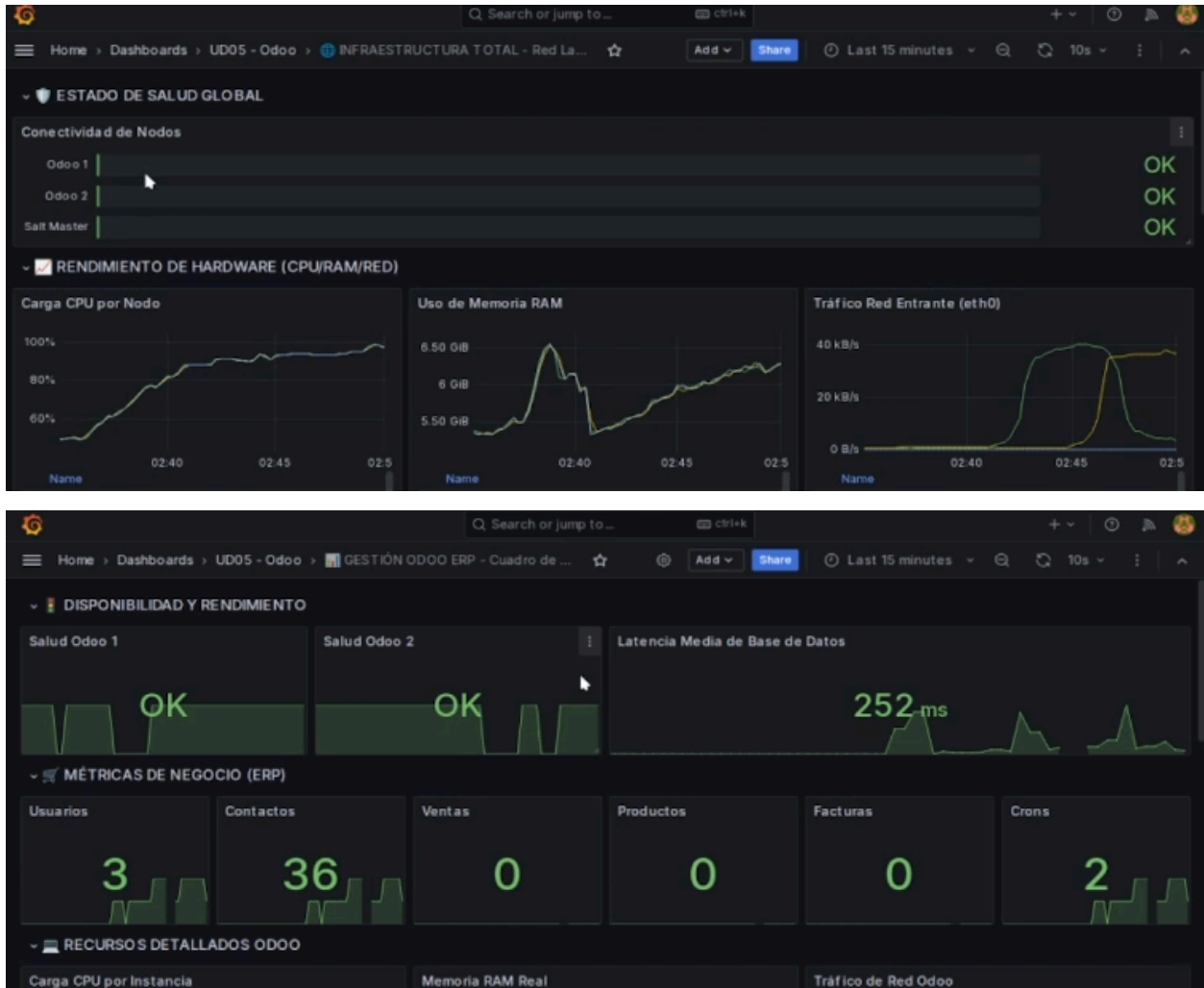


1. **Acceso a Grafana:** Abre tu navegador en <http://localhost:3000> (puerto estándar en la

configuración del laboratorio).

Usuario: **admin** / Contraseña: **admin** (a menos que hayas cambiado las credenciales en el **terraform.tf**).

2. **Dashboard:** En el menú lateral, ve a Dashboards y abre los distintos paneles.



Adaptación a Bajos recursos:

En la versión reducida, el dashboard es más sencillo y reducido.

Paso 6: (OPTATIVO) Ingeniería de caos (Simulación de incidentes)

Inyectamos fallos controlados para validar la resiliencia y la precisión de los dashboards:

```
bash scripts-laboratorio/07-simulate-odoo-incidents.sh
```

El script muestra un menú interactivo. Algunos de los escenarios a probar y visualizar en Grafana:

Opción	Efecto	Qué observar en Grafana
1 – Estrés de CPU	Inyecta un proceso md5sum infinito en Odoo 1.	La gráfica de "Carga CPU por Instancia" sube al 100% y se tiñe de rojo.
2 – Agotamiento de RAM	Reserva 1 GB de memoria dentro de Odoo 2 (solo si existe).	Salto vertical en la RAM; si supera el límite, el OOM Killer mata el contenedor y ves un reinicio.
3 – Inundación de logs	Genera 10.000 líneas de error (requiere Loki+Alloy).	El panel de logs en Grafana se llena de texto rojo.
4 – Muerte de proceso	Mata el proceso principal de Python de Odoo.	Blackbox Exporter detecta que la web no responde → panel de salud cambia a "CAÍDO".
5 – Limpieza	Detiene todos los procesos de estrés y restaura la normalidad.	Los paneles vuelven a valores normales.

Paso 7: Verificación integral del estado de todos los servicios (auditoría final)

Una vez que hayas realizado las pruebas de caos (o incluso después de un despliegue limpio), ejecuta este script para obtener un informe detallado de la salud de cada componente de la infraestructura:

```
bash scripts-laboratorio/03-check-status-odoo.sh
```

🧠 ¿Qué ocurre bajo el capó?

```
alumno@alumno-virtualbox: ~/Escritorio/Unidad-05-Integracion-Salt-Odoo/Laboratorio$ bash scripts-laboratorio/03-check-status-odoo.sh
ESTADO DE LA INFRAESTRUCTURA

Contenedores:
✓ ud05-salt-master
✓ odoo-pg1
✓ odoo-pg2
✓ odoo-minion-1
✓ odoo-minion-2
✓ odoo-prometheus
✓ odoo-grafana
```

El script no se limita a listar contenedores; ejecuta una serie de comprobaciones activas con reintentos y barras de progreso. Realiza las siguientes auditorías:

Sección	Qué comprueba	Cómo lo hace
 Contenedores	Que los contenedores esenciales están en estado Running.	docker ps y filtrado por nombres conocidos.
 Servicios web	Que Odoo, Grafana y Prometheus responden a peticiones HTTP.	curl con reintentos y espera (barra de progreso). Acepta códigos 200, 302, 500, 503 (considera que el servicio puede estar iniciando).

cAdvisor	Que la interfaz web de cAdvisor es accesible (sigue redirecciones).	curl -L contra http://172.28.0.52:8080/.
 Salt Stack	Que el proceso salt-master está en ejecución y muestra el número de Minions aceptados.	pgrep, ps aux dentro del contenedor y comando salt-key --list-all.
 PostgreSQL	Que la base de datos responde a consultas básicas.	docker exec odoo-pg1 pg_isready -U odoo.
 Exporters integrados en Odoo	Que node_exporter, odoo-exporter.py y salt-minion están corriendo dentro de odoo-minion-1.	pgrep dentro del contenedor.

Interpretación de los resultados:

- Todo en verde (✓) → la infraestructura está completamente operativa. Puedes pasar a la siguiente fase (dashboards o limpieza).
- Amarillo (⌚) → el servicio aún está iniciando (normal los primeros minutos después del despliegue). Espera un poco y vuelve a ejecutar el script.
- Rojo (✗) → el contenedor no se está ejecutando o el servicio no responde. Revisa los logs específicos con docker logs <nombre>.
- Minions aceptados: si aparece 0, es que no has ejecutado 04-salt-accept-keys.sh o las llaves no se han generado correctamente.

Adaptación a Bajos recursos:

En la versión reducida (un solo Odoo, sin Loki/Alloy), el script mostrará únicamente los contenedores que existen:

- La lista de contenedores no incluirá odoo-pg2, odoo-minion-2, odoo-loki ni odoo-alloy.
- Salt Stack reportará solo un Minion aceptado (odoo-minion-1).
- El resto de comprobaciones (Odoo, Grafana, Prometheus, cAdvisor, Blackbox, exporters) funcionarán igual.

Verificación final:

Este script te proporciona una fotografía completa de la salud del sistema. Guárdalo como herramienta de diagnóstico para futuros despliegues; si alguna vez algo falla, ejecútalo primero para identificar rápidamente la capa problemática (red, contenedor, servicio web, base de datos, Salt o exporters).

6. PRUEBAS MANUALES

Un ingeniero DevOps nunca asume que un despliegue ha sido exitoso solo porque la terminal no mostró errores. La infraestructura debe ser auditada activamente.

La prueba visual: el centro de mando

La confirmación definitiva del éxito del laboratorio es ver los datos en tiempo real. Abre tu navegador físico y visita los siguientes puntos de control:

El cuadro de mando (Grafana)

👉 <http://172.28.0.56:3000>

Credenciales: admin / admin (o las que hayas configurado en el terraform.tf).

Objetivo: Ve al menú lateral → Dashboards → abre el panel "GESTIÓN ODOO ERP - Cuadro de Mando". Si ves gráficas de CPU, RAM, métricas de negocio (usuarios, ventas) y logs en movimiento, has confirmado que el flujo de métricas (Prometheus) y logs (Loki) es correcto.

El motor de extracción (Prometheus)

👉 <http://172.28.0.51:9090>

Objetivo: Navega a Status > Targets. Todos los targets (infrastructure, odoo-exporter, blackbox, cadvisor, alloy) deben aparecer en color verde (UP). Esto valida que Prometheus puede raspar (scrapear) las métricas de cada componente sin errores de red o autenticación.

El microscopio de contenedores (cAdvisor)

👉 <http://172.28.0.52:8080>

Objetivo: Explora la interfaz de cAdvisor. Busca el contenedor odoo-minion-1 y observa su consumo de CPU, memoria y red. Esta herramienta te permite ver a nivel de kernel y cgroups cómo cada contenedor utiliza los recursos de la máquina anfitriona.

El archivo de logs (Loki) – solo versión completa

👉 Desde Grafana, en la sección Explore, selecciona la fuente de datos Loki y ejecuta la consulta {service="odoo-minion-1"}. Deberías ver aparecer las líneas de log del contenedor Odoo en tiempo real. Si ves texto, confirma que Alloy está recolectando correctamente la salida estándar.

7. OPERACIONES DE LIMPIEZA

Una vez finalizado el laboratorio, no basta con apagar los contenedores uno a uno. Para liberar por completo los recursos de tu equipo (puertos, memoria, volúmenes y red virtual) y dejar el sistema anfitrión en el mismo estado que antes de empezar, utiliza los scripts de limpieza incluidos en el proyecto. No ejecutes comandos sueltos como docker rm -f a mano – los scripts garantizan una demolición ordenada y evitan residuos.

Destrucción completa de la infraestructura

```
bash scripts-laboratorio/98-destroy-all.sh
```

¿Qué hace este script?

- Ejecuta terraform destroy -auto-approve para eliminar todos los contenedores, volúmenes persistentes y configuraciones creadas por Terraform.

- Borra los archivos de estado de Terraform (.tfstate*) para que tu próximo despliegue empiece desde cero.
- Elimina la red virtual interna LabCEFIRE (a menos que otros contenedores la estén usando).
- En la versión completa, destruye los 11 contenedores; en la versión de bajos recursos, solo los que corresponden (un Odoo, sin Loki/Alloy).

 **Tiempo estimado:** Entre 30 segundos y 1 minuto, dependiendo de cuántos contenedores haya que detener y eliminar.

Eliminación de la red puente (opcional)

Para eliminar la red puente creada:

```
bash scripts-laboratorio/99-destroy-network.sh
```

¿Qué hace este script?

- Verifica si la red LabCEFIRE todavía existe y la elimina de forma explícita.
- En caso de que la red estuviera siendo usada por algún contenedor huérfano, el script te informará y te sugerirá detenerlo primero.

Verificación final de limpieza

Para asegurarte de que no quedan procesos ni redes residuales, puedes ejecutar:

```
docker ps -a | grep -E "odoo|salt|prometheus|grafana|loki|alloy|cadvisor|blackbox"
docker network ls | grep LabCEFIRE
docker volume ls | grep ud05_
```

Si no aparece ningún resultado, la limpieza ha sido completa

8. PROBLEMAS COMUNES

En la ingeniería de sistemas, las infraestructuras complejas a veces fallan. Si te encuentras con un obstáculo durante el despliegue, revisa estos sospechosos habituales antes de pedir ayuda.

1. Colisión de puertos (Bind Error en Terraform)

Síntoma: Terraform muestra un error similar a Error: port is already allocated y los contenedores entran en un bucle de reinicio.

Causa: Los puertos 3000 (Grafana), 9090 (Prometheus) u 8069 (Odoo) ya están ocupados por otra aplicación en tu host (por ejemplo, un laboratorio anterior, una instancia local de Odoo o un IDE que usa el puerto 3000).

Solución:

- Detén el servicio conflictivo: **sudo lsof -i :3000** y mata el proceso con kill <PID> o cambia el puerto en el archivo **terraform.tf**.
- En la versión de bajos recursos, solo se exponen los puertos imprescindibles; la colisión es menos frecuente pero puede ocurrir igualmente.

2. Las métricas de negocio aparecen a cero o “No Data”

Síntoma: En Grafana, los paneles de Usuarios, Ventas y Contactos muestran valores 0 o No Data,

pero Odoo funciona correctamente.

Causas posibles:

- No se marcó la casilla "Demo Data" al crear la base de datos manualmente desde la web. El exporter necesita esos datos ficticios para mostrar valores.
- El script 03-init-odoo-db.sh no se ejecutó correctamente (error de conexión, falta de permisos).
- El contenedor odoo-minion-1 no tiene el exporter corriendo (revisa docker exec odoo-minion-1 pgrep -f odoo-exporter.py).

Solución:

- Vuelve a ejecutar el script 03-init-odoo-db.sh (asegurándote de que la base de datos se crea con demo).
- Si usaste la interfaz web, crea una nueva base de datos marcando explícitamente "Load demonstration data".
- En bajos recursos, recuerda que solo existe un nodo; el script adaptado ya inicializa el nodo único correctamente.

3. Timeout en la conexión del Salt Master o Minions que no responden al test.ping

Síntoma: Al ejecutar 05-salt-test-ping.sh no obtienes ningún True o el comando se queda colgado.

Causa principal: No se aceptaron las llaves RSA de los Minions. Salt opera con Zero Trust; el Master ignora cualquier comunicación no autorizada.

Solución:

- Ejecuta **04-salt-accept-keys.sh** y verifica que aparecen llaves aceptadas con **salt-key -L**.
- Si los Minions existen pero no responden, reinicia el servicio salt-minion dentro del contenedor: **docker exec odoo-minion-1 systemctl restart salt-minion**.
- En bajos recursos, solo hay un Minion; asegúrate de que el contenedor odoo-minion-1 tiene instalado salt-minion (en la versión low-spec, por diseño, puede que no se instale para ahorrar recursos; en ese caso, el paso de Salt es meramente ilustrativo y puedes omitirlo).

4. Loki no muestra logs o Alloy falla al iniciar

Síntoma: El panel de logs en Grafana está vacío (versión completa) o Alloy se reinicia constantemente.

Causa frecuente: Falta el volumen del socket de Docker o permisos insuficientes. Alloy necesita montar `/var/run/docker.sock` y leer los logs de los contenedores.

Solución:

- Verifica que el contenedor odoo-alloy tiene montado el socket: **`docker inspect odoo-alloy | grep -A5 Mounts`**.
- Comprueba que el usuario bajo el que corre Alloy tiene permisos de lectura sobre el socket (en la práctica, suele correr como root en el laboratorio).
- Nota: En la versión de bajos recursos, este problema no aparece porque Loki y Alloy no están desplegados (sección eliminada).

5. El contenedor Odoo no arranca por falta de memoria (OOM Killer)

Síntoma: El contenedor odoo-minion-1 aparece como exited o se reinicia cada pocos segundos, y en los logs del sistema ves mensajes sobre Out of memory.

Causa: Tu equipo anfitrión no tiene suficiente RAM para el límite asignado (por ejemplo, 1.5 GB para Odoo más el resto de contenedores).

Solución:

- Reduce aún más la RAM de Odoo en el **`terraform.tf`** (por ejemplo, a 768 MB) y vuelve a desplegar.
- En bajos recursos, el límite ya está en 1024 MB; si el problema persiste, cierra otros programas o reduce a 768 MB.